

VB Predictor — Teljes körű Projekt Audit & Vezetői Jelentés

Dátum: 2026-07-07
Auditor: Claude Code (mentor + business analyst üzem módban)
Státusz: 🟡 **Működik, de égetően szükséges javításokkal**

1. VEZETŐI ÖSSZEFOGLALÓ

Mi működik jól ✅

Terület	Státusz	Részletek
Adatbázis	🟢 11037 meccs, 243 csapat, 2080 játékos	Széleskörű adatgyűjtés, SQLite WAL módban
ELO rendszer	🟢 43792 log bejegyzés, 55 static_elo	Mindkét ELO rendszer működik
Csoportkör	🟢 Teljesen lefedett	72/72 meccs eredménnyel, 12 csoport
Napi predikciók	🟢 64 kiértékelt tipp	53.1% pontosság, +7.7% kumulált ROI
Feature engineering	🟢 18 feature, TimeSeriesSplit CV	Jól felépített pipeline

Mi fáj igazán 🔴

Terület	Státusz	Részletek
Pipeline elavult	🔴 5 napja nem futott	Utolsó: július 1., ma július 7. van
R32 hiányos	🟡 8/16 meccsnek nincs eredménye	Júl 1-3-i meccsek hiányoznak
R16+ nincs betöltve	🔴 Nincs R16, negyeddöntő, elődöntő, döntő	A torna lényegében a R32-nél megállt
Szimuláció törött	🔴 Knockout bracket nem oldható fel	A DB-ben valós csapatnevek, a kód slot-neveket vár
Döntetlen predikció	🔴 Recall=0 a 4. és 5. foldban	A modell gyakorlatilag sosem tippel döntetlent
Transzparencia	🟡 Train-serving skew	A modell elo_log-on tanul, de static_elo-val prediktál

Business mutatók 

Metrika	Érték	Értékelés
Modell pontosság (CV)	57.9% (±2.4%)	🟡 Elfogadható (baseline 33.3%)

Metrika	Érték	Értékelés
Valós tipp pontosság	53.1% (34/64)	🟡 Alulmúlja a CV-t, de pozitív
Kumulált ROI	+7.7%	🟢 Pozitív, de vékony
Nettó profit	+\$49.30	🟡 \$640 tét mellett
Utolsó pipeline futás	6 napja	🔴 Elavult, adatvesztés kockázata
Teszt lefedettség	0%	🔴 Nincsenek tesztek

2. KRITIKUS HIBÁK (Azonnali javítás)

🔴 Hiba #1 — Pipeline leállt, adatok elavultak

Hol: `run_daily.sh` → `update_results.py`, `evaluator_daily.py`, `daily_predictor.py`

Hatás: A napi pipeline utoljára július 1-én futott le. 8 R32 meccs eredménye hiányzik (júl 1-3). Nincs R16+ adat betöltve.

Valós következmény: Ha most megnézed a predikcióidat, a legtöbb "PENDING" státuszban van. A ROI számítás is régi.

Gyökérok: Valószínűleg vagy az API kulcs járt le, vagy a `LEAGUE_ID = 1` nem helyes a 2026-os VB-re az API-Footballon.

Javítás menete:

- Ellenőrizd az API kulcsokat a `.env` fájlban
- Futtasd manuálisan: `python scripts/update_results.py --date 2026-07-02` (majd 03, 04...)
- Ha az API nem ad adatot: a football-data.org fallback-nek kellene dolgoznia
- Miután a R32 eredmények megvannak, kell egy szkript ami betölti a R16 mérkőzéseket a JSON-ból
- Állíts be monitoringot: ha a pipeline nem fut le, kapj emailt/értesítést

Sürgősség: KRITIKUS — a VB már a kieséses szakaszban jár, és nincs naprakész adat.

🔴 Hiba #2 — A knockout bracket nem oldható fel a szimulációban

Hol: `modell/simulation.py` → `resolve_r32_bracket()`

Hatás: A szimuláció teljesen használhatatlan a kieséses szakaszra.

Gyökérok: A `resolve_r32_bracket()` függvény slot-neveket vár ("1A", "2B", "3A/B/C/D/F"), de az adatbázisban VALÓS csapatnevek vannak a R32 mérkőzéseknél (pl. "South Africa", "Canada"). A kód emiatt minden párosításnál `None`-t kap, a R32 lista üres lesz, és a teljes knockout szakasz kimarad.

```
# simulation.py ~260. sor
# Ezt várja: "1A" → slot_to_team["1A"] = csapat
# De a DB-ben: "South Africa" → slot_to_team.get("South Africa") = None ← BAJ!
```

Javítás: A R32 bracket template-t slot-nevekkel kell betölteni, vagy a `resolve_r32_bracket()` függvényt kell átírni, hogy valós csapatneveket kezeljen.

● Hiba #3 — `api_football_loader.py` 2022-re van hardcode-olva

Hol: `data/api_football_loader.py` 30. sor

Hatás: A teljes API-Football integráció a 2022-es VB adatait tölti be, nem a 2026-ost!

```
# 30. sor – EZ BAJOS:  
WC_SEASON = 2022 # ← 2026 kellene!
```

Valós következmény: A `load_squads()` függvény a 2022-es kereteket hozza le, ami teljesen félrevezeti a játékos-statisztikákat. 2026-ban más játékosok, más keretek.

● Hiba #4 — Döntetlen predikció recall=0

Hol: `modell/train.py`, `cv_report.txt`

Hatás: A modell a 4. és 5. keresztvalidációs foldban **egyetlen döntetlent sem jósol be**. A draw recall=0.00. Ez azt jelenti, hogy a modell a későbbi meccseken (amik a legfrissebbek és legrelevánsabbak) teljesen vak a döntetlenre.

Business hatás: A fogadási piacon a döntetlen szorzók a LEGMAGASABBAK (gyakran 3.0-4.0). Ha a modell soha nem tippel döntetlent, kihagyja a legnagyobb értékű tetteket. Ez évi több száz dollárnyi kihagyott profit.

Gyökérok: Class imbalance — a döntetlen a legritkább kimenetel (~22%). Az XGBoost + isotonic kalibráció nem kezeli jól a minor osztályt.

Javasolt megoldások:

- `scale_pos_weight` vagy `class_weight` paraméter az XGBoost-ban
- SMOTE oversampling a döntetlen osztályra
- Külön döntetlen-detektor modell (meta-learner)
- A döntetlen alapértelmezett valószínűségének növelése post-hoc (Bayesian correction)

● Hiba #5 — Tranzakciókezelési rés `update_results.py`-ban

Hol: `scripts/update_results.py` → `_update_elo()`

Hatás: A `_update_elo()` függvény saját `conn.commit()`-et hív a `update_match_results()` try/except blokkján BELÜL.

```
def _update_elo(conn, ...):  
    cursor.execute(...)  
    conn.commit() # ← ITT COMMITOL, függetlenül a külső tranzakciókezeléstől!  
    ...  
  
def update_match_results(conn, ...):  
    try:  
        for fixture in fixtures:  
            ...  
            _update_elo(conn, ...) # ← belső commit!  
        conn.commit() # ← külső commit
```

```
except Exception as e:
    conn.rollback() # ← MÁR KÉSŐ! A belső commit már véglegesített!
```

Következmény: Ha a 3. meccs feldolgozásánál hiba történik, az 1. és 2. meccs ELO frissítése MÁR el van mentve. A rollback csak a 3. meccset vonja vissza. Ez inkonzisztens adatbázisállapotot eredményez.

Megoldás: Távolítsd el a `conn.commit()`-et a `_update_elo()`-ből. A commitot csak a `update_match_results()` végezze.

3. KÖZEPES SÚLYÚ HIBÁK

🟡 Hiba #6 — Tournament ID összemosás

Hol: `data/intl_results_loader.py` → `_get_or_create_tournament()`

Probléma: Minden mérkőzés az év alapján kap `tournament_id`-t, függetlenül a tényleges tornától:

```
row = conn.execute("SELECT id FROM tournament WHERE year=?", (year,)).fetchone()
if row:
    return row["id"] # ← 2022-es WC és 2022-es barátságos meccs ugyanaz a
tournament!
```

Hatás: A 2026-os tornába bekerültek selejtező mérkőzések (32 meccs "Qualifier" stage-dzsel). A 2026-os csoportmérkőzések (72) és a R32 (16) is ugyanahhoz a `tournament_id`-hoz tartoznak. Ez rövidtávon nem okoz gondot, de ha valaha tornánkénti elemzést akarsz, fals adatokat kapsz.

🟡 Hiba #7 — "L101"/"L102" placeholder csapatok az adatbázisban

Hol: `data/worldcup_json_loader.py` + adatbázis `team` tábla

Probléma: A JSON betöltő csak a "W" prefixű helyőrzőket szűri ki:

```
if t1_name.startswith("W") and t1_name[1:].isdigit():
    stats["skipped"] += 1
    continue # ← "W101" kiszűrve, de "L101" nem!
```

Hatás: Az "L101" (ID=250) és "L102" (ID=251) csapatok bekerültek a `team` táblába. A "Match for third place" meccs ezekkel a nemlétező csapatokkal van összekötve. Ha valaki lefuttat egy "all teams" listát, szellem csapatokat lát.

🟡 Hiba #8 — 50+ bracket placeholder csapat az adatbázisban

Hol: Adatbázis `team` tábla

Probléma: A R32 bracket slot-nevek ("1B", "1C", "2A", "2L" stb.) csapatként lettek létrehozva, de sosem lettek használva.

Hatás: A `team` táblában 243 csapat van, de ebből csak ~48 a tényleges 2026-os VB résztvevő. A többi vagy placeholder, vagy történelmi csapat. Ez félrevezető lehet adatelemzéskor.

🟡 Hiba #9 — "Bosnia and Herzegovina" vs "Bosnia-Herzegovina" duplikáció

Hol: Adatbázis `team` tábla

Probléma: Ugyanaz az ország két néven szerepel:

- "Bosnia and Herzegovina" (ID=59, code=BOS, 130 meccs)
- "Bosnia-Herzegovina" (ID=216, code=BOS1, 3 meccs)

Hatás: A 3 meccs "Bosnia-Herzegovina" néven nem kapcsolódik össze a "Bosnia and Herzegovina" statisztikákkal. Az ELO számítás külön kezeli őket.

🟡 Hiba #10 — `requirements.txt` sérült

Hol: `requirements.txt`

Probléma: A fájl null byte-okkal előtagolt — vélhetően egy "UTF-8 with BOM" vagy bináris írás okozta.

4. ARCHITEKTURÁLIS FIGYELMEZTETÉSEK

⚠️ Train-serving skew (tanítás-élesítés eltérés)

A probléma: A modell **betanítása** az `elo_log` tábla dinamikus ELO értékein történik. De az **éles predikciók** a `static_elo` tábla importált (elratings.net) értékeit használják.

```
# pipeline.py build_training_matrix(): elo_log-ból
elo_home = elo_tl.get((home_id, mid), 1500.0) # ← dinamikus ELO

# pipeline.py build_prediction_row(): static_elo-ból
h_stat = conn.execute("SELECT elo_rating FROM static_elo WHERE team_id=?", ...) #
← importált ELO
```

Miért probléma? A modell soha nem látott `static_elo` értékeket tanítás közben. Ha a két ELO rendszer eltér (és eltér: pl. Spanyolország statikus 2165, de a dinamikus lehet más), a modell rosszul kalibrált valószínűségeket ad.

Hatás a businessre: Ha megkérdezi valaki, hogy "mi alapján jósol a rendszer", nem tudsz koherens választ adni. A befektetők/elemzők számára ez megkérdőjelezi az egész megközelítést.

Javaslat: Válaszd ki az EGYIK ELO rendszert, és használd következetesen tanításra és predikcióra egyaránt. A `static_elo` egyszerűbb és pontosabb a valósághoz, de az `elo_log` dinamikusabban követi a formát.

⚠️ Nincsenek tesztek

A teljes projektben **0 teszt** található. Nincs `test_*.py`, nincs `pytest`, nincs `unittest`. Ez egy ML+betting alkalmazásnál, ahol pénz forog kockán, elfogadhatatlan.

Minimum amit el kellene várni:

- Unit tesztek a pipeline függvényekre (főleg `build_prediction_row`, `_build_form_cache`)
- ELO számítás verifikáció (ismert input → ismert output)

- Tranzakciókezelés tesztelése (commit/rollback scenáriók)
- A szimuláció konzisztencia tesztjei (pl. 1000 szimulációból mindig 48 csapat, 12 csoport)

⚠ Nincs monitoring/alerting

Ha a napi pipeline elszáll:

- Nem tudsz róla (nincs email, nincs webhook, nincs Slack)
- Az adatok elavulnak
- A predikciók nem készülnek el
- A ROI számítás megáll

Egy pénzügyi alkalmazásnál, ahol napi döntések függnék a rendszertől, ez egy üzleti kockázat.

5. A PIPELINE JELENLEGI ÁLLAPOTA (ADAT INTEGRITÁS)

Adat	Darab	Státusz
Csoportkör mérkőzések	72/72 eredménnyel	✅ Teljes
R32 mérkőzések	8/16 eredménnyel	🟡 Hiányos
R16+ mérkőzések	0	🔴 Nincs betöltve
Selejtező mérkőzések (2026)	32	⚠ Idegen a tornában
"Harmadik hely" meccs	1 (de L101 vs L102)	🔴 Placeholder
Utolsó pipeline futás	Július 1., 2026	🔴 6 napja

A 2026-os VB a valóságban valószínűleg már a R16-nál vagy a negyeddöntőknél tart. A rendszered adatai 5+ napja nem frissültek. Ez azt jelenti, hogy a predikciók és a ROI számítás nem tükrözi a valóságot.

6. ÜZLETI AJÁNLÁSOK

💰 Azonnali pénzügyi javaslatok

- Tét optimalizálás:** Jelenleg fix \$10 tétet használsz minden meccsre. Ez nem optimális. Javasolt:
 - **Kelly Criterion** (bankroll növekedés maximalizálása)
 - Bankroll management: ha \$1000 a kereted, ne \$10/tét legyen, hanem a Kelly által javasolt összeg
- Döntetlen stratégia:** A döntetlen szorzók általában 3.0-4.0 között vannak. Ha a modell predict_draw > 25%, a várható érték már pozitív lehet. Ez a legalacsonyabban lógó gyümölcs.
- Value bet detektálás:** A `daily_predictor.py` már számol EV-t, de nem szűri ki a negatív EV-t. Jelenleg MINDEN meccsre beteszelsz \$10-et, függetlenül a várható értéktől. Ez nem stratégia, hanem szerencsejáték.

📈 Stratégiai javaslatok

4. **A/B tesztrendszer:** Hozz létre egy keretrendszert, ahol a régi modell és az új modell párhuzamosan megy, és összehasonlítod a ROI-t. Csak akkor váltasz, ha az új jobb.
5. **Adat verziózás:** Minden `simulation_predictions.json` mentés tartalmazzon timestamp-et a fájlnevben. Jelenleg felülírod a régit.
6. **Klubformák bekötése:** A `club_form_loader.py` már le van írva, de sosem lett használva a modellben. A játékosok klubformája (utolsó 10 meccs, percek, form) jelentős prediktív erővel bírhat. Ez egy ingyen elérhető adatforrás.
7. **Poisson-gólmodell:** A jelenlegi W/D/L modell helyett egy Poisson-regresszió pontosabb lenne a gólok becslésére, ami pontosabb döntetlen valószínűséget adna. Kombinálva az XGBoost-tal egy hibrid rendszer (Poisson a gólokra, XGBoost a győztesre) lehet az iparági színvonal.

Kockázatkezelés

8. **API kulcs lejárat kezelés:** Jelenleg ha az API kulcs lejár, a pipeline csöndben elbukik. Kell egy megbízható API kulcs rotáció vagy többszintű fallback.
9. **Adatbázis backup:** Az SQLite fájl (5.3 MB) biztonsági mentés nélkül tárolod. Ha megsérül, minden adat elveszik. Napi automatikus backup kell.
10. **Naplózás:** A `print()` helyett `logging` modul kell. A stdout nem perzisztens, nem kereshető, nem súlyozható (info/warning/error).

7. FEJLESZTŐI JEGYZETEK (Kód minőség)

Kód antifattern-ek

Pattern	Hol	Javaslat
<code>from pipeline import ...</code>	<code>simulation.py:28</code>	<code>from modell.pipeline import ...</code> (TODO.txt-ben is szerepel)
"JAVÍTÁS" kommentek	<code>pipeline.py:126</code>	Távolítsd el az ideiglenes kommenteket
Varázsszámok	<code>pipeline.py:139</code> → <code>return 150</code>	Konstansba kell
Exception-elnyelés	<code>simulation.py:85</code> → <code>except Exception: pass</code>	Legalább logolni kell
Fix tét konstans	<code>daily_predictor.py:27</code> → <code>STAKE = 10.00</code>	Konfigurációba kell

Duplikált kód

- `get_connection()` import minta: majdnem minden fájl elején `sys.path.insert(0, ...) + from data.db import get_connection`
- ELO expected score számítás: külön implementáció `elo.py`-ban, `simulation.py`-ban, `daily_predictor.py`-ban és `update_results.py`-ban (4 helyen!)

- Csapatnév mapping: `WORLDCUP_JSON_LOADER.py`, `intl_results_loader.py`, `update_results.py`, `daily_predictor.py` — mind külön tartják karban
-

8. KONKRÉT TEENDŐK (Prioritási sorrendben)

P0 — Ma este megcsinálni

1. Futtasd le manuálisan: `python scripts/update_results.py --date 2026-07-02` (07-03, 07-04...)
2. Javítsd ki a `_update_elo()` commit bugját
3. Ellenőrizd az API kulcsokat

P1 — Holnap

4. Javítsd ki a `api_football_loader.py` WC_SEASON = 2026-ra
5. Írj egy szkriptet a R16+ knockout mérkőzések betöltésére
6. Távolítsd el a placeholder csapatokat (L101, L102, 1B, 1C, 2A...)

P2 — Ezen a héten

7. Töröld a "Bosnia-Herzegovina" duplikátumot (merge a 3 meccset)
8. Kelly Criterion implementálása a tét optimalizáláshoz
9. `requirements.txt` javítása
10. Structured logging bevezetése

P3 — Ebben a hónapban

11. Tesztek írása (pipeline, ELO, tranzakciókezelés)
 12. Train-serving skew megszüntetése (static_elo vs elo_log)
 13. Döntetlen osztály javítása (SMOTE, class_weight)
 14. Poisson-gólmodell bevezetése
 15. Adatbázis backup + monitoring
-

9. ZÁRSZÓ

Röviden: Van egy szilárd alapokon nyugvó rendszered, ami 53%-os pontossággal és +7.7%-os ROI-val működik. Ez jobb, mint a legtöbb hobbi ML-projekt. **De** a rendszer jelenleg elavult adatokkal dolgozik, a kieséses szakasz szimulációja törött, és a döntetlen osztály gyakorlatilag nem létezik.

A legnagyobb problémád **nem a kódminőség** — az rendben van egy egyszemélyes projekthez. A legnagyobb problémád az, hogy **a pipeline leállt, és senki nem vette észre**. Egy betting rendszernél, ahol napi döntések múlnak az adatokon, ez a #1 kockázat.

Ha a fenti P0 és P1 javításokat megcsinálod, a rendszered újra üzemképes. A P2 és P3 itemek a különbség a "jó hobbi projekt" és a "valódi pénzt termelő rendszer" között.